

ANCHOR: Asset-Informed Geometric Skills for Language-Conditioned Manipulation

Anonymous Authors

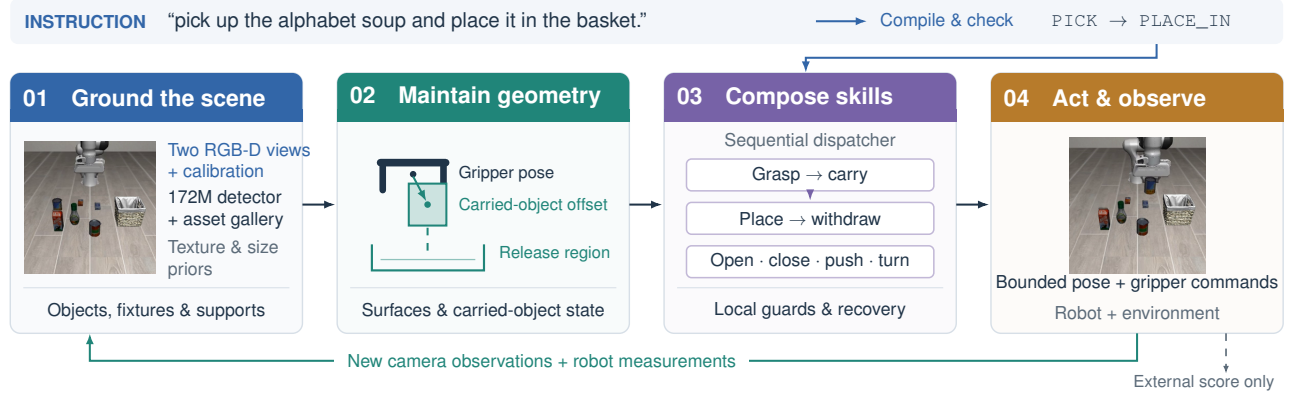


Fig. 1. From language to physical interaction with ANCHOR. Supported instructions specify a skill sequence; calibrated RGB-D and asset priors provide the geometry used to execute it. The controller carries object and fixture state across stages and advances using local observation checks. The frozen 172M-parameter detector is the only neural model. Insets show the recorded Object 00 rollout; the geometry illustration is schematic. External benchmark success is recorded separately from policy execution.

Abstract—Multi-stage manipulation requires each skill to leave geometry and contact conditions that the next skill can use. We present ANCHOR, an asset-informed system that carries accepted scene estimates, grasp-dependent object offsets, and fixture anchors between geometric skills. A deterministic compiler maps supported instructions to a skill sequence, while local checks on robot measurements and selected visual updates govern execution. One frozen 172M-parameter detector, two calibrated RGB-D views, and public asset priors supply the perceptual inputs. Manually designed controllers generate actions without robot-demonstration training. In a full evaluation of 400 fixed initializations spanning 40 LIBERO tasks, one frozen controller reaches the official success predicate in 376 episodes (94.0%). Separate controller and evaluator outcomes disagree in 41/400 episodes, exposing limitations of local completion checks. The study characterizes geometric execution under known-asset and calibrated-sensing assumptions.

I. INTRODUCTION

Consider the instruction to place a mug inside a microwave and close it. The mug must enter the cavity in its grasped configuration, separate from the fingers on release, and remain inside as the hand withdraws and engages the door. A correct object label and skill sequence leave these physical transitions unresolved. A mug caught on a finger can be pulled out by retreat; a drawer opened successfully can leave the arm poorly configured for the next grasp. Multi-stage execution therefore depends on both the goal of each skill and the state it leaves for the next one.

Existing approaches connect language to action through several interfaces. Vision-language-action (VLA) models learn action prediction from robot demonstrations [1]–[3]. Program-based systems expose perception and control

calls [4], while geometric approaches express desired behavior through spatial value maps or keypoint constraints [5], [6]. We study the execution interface in a constrained setting with known assets, supported language forms, and calibrated sensing. The question is how to connect perceptual estimates to the geometric targets and transition checks needed across successive skills, and how those explicit quantities help diagnose observed failures.

We present ANCHOR (Fig. 1), a language-conditioned tabletop manipulation system built around three kinds of execution state. Observed surfaces and asset dimensions determine contact and placement regions. A carried-object offset converts an object goal into an end-effector target. Local checks on pose, gripper width, visual geometry, and contact evidence govern progression between stages. A deterministic compiler supplies the skill sequence, while a frozen 172M-parameter Grounding DINO detector and an asset gallery ground objects in two calibrated RGB-D views. The system generates actions through manually designed controllers, with no robot-demonstration training of an action policy.

We evaluate 40 tasks from LIBERO’s Spatial, Object, Goal, and Long suites [7], with ten official initial states per task and one frozen controller. This system study contributes:

- 1) An asset-informed execution interface that connects accepted scene estimates, grasp-dependent offsets, and fixture anchors to the targets and transition checks of manipulation skills.
- 2) A complete evaluation on 40 tasks, with declared sensing and prior knowledge, an audited neural model count, and reproducible episode records.

- 3) An analysis of phase traces, failure locations, and stopping outcomes that identifies mismatches between local completion and the benchmark predicate in the evaluated controller.

II. RELATED WORK

A. Geometric Representations for Manipulation

Task and motion planning couples discrete action choices with continuous geometric feasibility [8]. kPAM uses semantic 3-D keypoints to specify manipulation goals as geometric costs and constraints [9]. VoxPoser grounds language in spatial value maps for motion planning [5], and ReKep optimizes sequences of relational keypoint constraints [6]. ANCHOR shares the use of explicit geometry between perception and action. Its representation consists of object bounds, support and fixture geometry, and carried-object offsets, consumed by specialized finite-state skills. This fixed skill structure defines its coverage; it does not optimize a general task-and-motion problem.

B. Language and Executable Skills

SayCan combines language-model proposals with learned skill affordances to select executable actions [10]. Code as Policies generates programs that process perceptual outputs, express control logic, and call robot interfaces [4]. These systems use language models to construct behavior. ANCHOR binds spatial references to observed entities using a deterministic grammar and a checked skill vocabulary. The grammar fixes language coverage; our study concerns execution and the state passed between its skills.

C. Pretrained Perception and Learned Action Policies

Grounding DINO provides text-conditioned object proposals [11]. ANCHOR combines its frozen detector with calibrated depth and asset-derived texture and size priors. OpenVLA [1] and SmolVLA [2] learn robot action generation from demonstrations, with SmolVLA emphasizing a compact architecture. OpenVLA-OFT studies design choices for faster and more successful VLA execution [3]. Section V-E compares their model requirements and reported outcomes with ANCHOR, accounting for its asset knowledge and manual skill development.

III. METHOD

ANCHOR compiles an instruction, grounds its entities, and executes the resulting skills in order (Fig. 1). Each phase constructs a pose and gripper target from the available geometry and uses local checks to continue, advance, retry, or terminate. Table I declares the available information; Table II describes the state passed through this execution interface.

A. Task and Information Interface

Given an instruction ℓ , the controller produces actions $a_t = \pi(o_{\leq t}, \ell; \mathcal{P})$, where $\mathcal{P}$ contains fixed prior information and the observation at step t is

$$o_t = \{I_t^c, D_t^c, K^c, T_{WC}^c\}_{c \in \{a, w\}}, r_t\}. \quad (1)$$

TABLE I
DECLARED CONDITIONS FOR THE EVALUATED SYSTEM.

Property	Setting
Policy observations	Fixed and wrist RGB-D, camera calibration, robot measurements
Offline priors	Public asset textures/dimensions, robot geometry, hand-written skill rules
Action generation	Geometric state machines and clipped Cartesian increments
Neural model	One frozen Grounding DINO Tiny (172M parameters)
Action-policy training	None; skills and thresholds are designed manually
Scene truth in policy	No object pose labels, segmentation IDs, or BDDL predicates in observation types
Success feedback	External latch; no score-driven policy stopping
Coverage	Known benchmark assets and implemented instruction grammar

Here a and w denote the fixed and wrist cameras, I is RGB, D is metric depth, K is calibration, and T_{WC} maps camera coordinates into the world frame. Robot measurements r_t include end-effector pose, joint state, gripper state, and available force/torque measurements. The grasp-place adapter uses pose, joint positions, and gripper width; contact execution can access the broader robot measurement interface.

Depth, camera calibration, and robot measurements are supplied by the simulator adapter. The policy observation types exclude object pose labels, segmentation IDs, BDDL predicates, reward, and task-success flags. Task identifiers and initial-state indices remain with the evaluator; reset passes the instruction and a nonsemantic episode token.

The prior collection $\mathcal{P}$ contains object texture histograms, dimensions derived from public collision geometry, robot geometry, class-specific handling rules, and motion thresholds. These priors specify how to interpret observations and parameterize skills. The neural detector is pretrained; the action controller is designed manually. No action-policy fitting or robot-demonstration loading occurs in the evaluation.

B. Constrained Language Compilation

The compiler maps supported language to

$$\mathcal{G}(\ell) = (s_1, \dots, s_M), \quad s_j = (k_j, e_j^{\text{src}}, e_j^{\text{dst}}, \rho_j). \quad (2)$$

Here k_j is a skill, the entity bindings specify source and destination, and ρ_j represents a placement relation or instance selector. The vocabulary contains pick, place-on, place-in, relative placement, stack, open, close, push, turn-on, and turn-off. For example, opening a drawer and placing a bowl inside produces an articulation segment followed by pick and place-in steps. A two-object basket instruction produces two successive pick/place pairs.

Bindings retain relations such as *next to*, *between*, or an ordering among visible instances. They do not encode simulator object identity. Before motion, the compiler checks skill arguments and held-object consistency, including whether each place step refers to the object acquired by a preceding pick. Physical executability is assessed during skill execution. The grammar covers known LIBERO phrases and an

TABLE II
GEOMETRIC STATE AND ITS USE DURING EXECUTION. UPDATES ARE
SELECTED BY THE ACTIVE SKILL AND PHASE.

State	Source or update	Use
Scene estimates	RGB-D grounding with texture and size priors	Entity binding, approach and placement regions
Carried offset $\hat{d}_t$	Grasp geometry; selected RGB-D refreshes	Object goal to end-effector target, Eq. (5)
Fixture anchors	Visible support/fixture fits and contact-skill outputs	Subsequent placement and fixture interaction
Phase and counters	Local checks and executed actions	Advancement, bounded recovery, stopping

allow-listed entity vocabulary; no language-model resolver is configured.

C. State across Skill Boundaries

The execution state links the current skill and phase to accepted scene geometry, a carried-object offset, and fixture anchors (Table II). Robot pose and gripper measurements support the servo at every action step. Visual grounding is invoked at selected stages; observing a new frame does not imply re-estimating every target. Some refresh routines compare new estimates with the stored geometry to reject changes caused by hand occlusion.

This persistence matters across interactions. Placement consumes the offset established by the grasp; a placement after drawer opening consumes its observed drawer anchor; sequential stove placements share a support estimate obtained while the burner is unoccluded. The stored quantities make these dependencies explicit, while their update rules determine when new evidence can revise a target.

D. Asset-Informed Visual Grounding

The deployed Grounding DINO Tiny checkpoint contains 172,249,090 unique parameters, including its visual and text encoders. All weights are frozen, and one detector instance serves both camera views in each evaluation process. The asset gallery, language compiler, and geometric controllers add no neural model parameters. The pinned checkpoint and parameter audit are included in the manuscript artifacts.

Each calibrated pixel $\tilde{u} = (u, v, 1)^\top$ with valid depth contributes

$$x_W = R_{WC}(D(u, v)K^{-1}\tilde{u}) + p_{WC}. \quad (3)$$

Here R_{WC} and p_{WC} are the rotation and translation of T_{WC} . Workspace filtering, support estimation, and connected components produce candidate point sets. Grounding DINO supplies additional language-conditioned boxes, which are converted to 3-D hypotheses using depth. These proposals are combined with observed bounds and asset-derived texture and size evidence. The gallery compares normalized HSV histograms using

$$b(h, h_k) = \sum_i \sqrt{h_i h_{k,i}}. \quad (4)$$

Its classification score combines color and shape terms, with configured weights 0.4 and 0.6, a softmax temperature of

0.13, and additional class-dependent footprint gates. This combination associates visual proposals with known asset classes without an oracle object mask.

Cross-camera hypotheses with matching labels are clustered in 3-D. For a cluster, the implemented fusion weights are proportional to $\max(q_i, 0.05)\sqrt{n_i}$, where q_i is confidence and n_i is point count. The default center-distance threshold is 4.5 cm. The resulting state distinguishes observed bounds and supports from geometry completed using size priors. A prior can stabilize the center of a partly visible object without establishing that an unobserved region is free.

Geometric selectors bind instruction references to candidate instances. At selected phase transitions, fresh views are checked against previous estimates to reject abrupt changes caused by hand occlusion. Nearby repeated objects can still produce ambiguous associations. Fixture routines further refine the selected geometry into support surfaces and opening regions.

E. Carried-Object Geometry and Placement

Consecutive manipulation skills share a grasp-place controller. Its phases include acquisition, pregrasp motion, approach, finger closure, retention checking, lift, transport, descent, release, and retreat. Transitions combine pose tolerances, measured gripper width, visual evidence, and bounded retry logic. The evaluated configuration uses exterior pinch and closing rim pinches. Object-class rules select the nominal mode; observed geometry determines the grasp target and available retries.

The chosen grasp also determines the geometry used during placement. In particular, the gripper and carried object generally have different centers. Let $\hat{d}_t$ be the estimated world-frame vector from the end effector to the carried-object reference point. For desired object location p_o^* , placement uses

$$p_{ee}^* = p_o^* - \hat{d}^*, \quad \hat{d}^* = \Delta R \hat{d}_t. \quad (5)$$

Here $\Delta R = R_{ee,t}^* R_{ee,t}^\top$ is the planned wrist rotation in the world frame. The relation assumes that the object remains fixed relative to the gripper over this motion; slip or partial release can invalidate that estimate. The implementation supplements Eq. (5) with support, release-height, aperture, and fixture-specific corrections. The offset is estimated from precontact geometry or refreshed RGB-D evidence; its origin is retained in the execution trace.

Placement must also leave room for the hand to withdraw. Microwave entry aligns the carried object with the cavity width and uses a lateral approach. Repeated basket placements use separate diagonal landing centers. Before basket withdrawal, the controller checks measured jaw clearance and can apply a bounded wrist tilt to clear a tall object near the palm. Release is followed by a geometric observation check before the program advances.

For microwave release, retreat waits for a measured 78-mm jaw opening and a settling dwell, because a commanded opening alone does not establish that the mug rim has cleared the fingers.

F. Fixture Geometry and Contact Skills

The dispatcher separates consecutive grasp–place steps from atomic contact skills. Drawer and door interactions use observed fixture geometry to construct approach, engagement, motion, release, and retreat stages. Knob and push skills use their own target detectors and motion templates. Shared episode state includes an initial robot-pose reference and, where applicable, an observed drawer anchor for subsequent placement.

Grounded object boxes are refined into surfaces needed for physical interaction. A complete visible burner disk anchors the stove support center; public plate thickness supplies the offset to the load-bearing surface. That unoccluded estimate is retained across sequential placements. For an open microwave, visible vertical control-panel and side-wall surfaces, together with the roof height, constrain the cavity and hinge frame through public physical dimensions. A flat tabletop crop or a roof without supporting vertical structure is insufficient for this fit.

For microwave closure, the skill fits a vertical door panel within the observed hinge sweep. Initial fitting requires panel evidence outside the appliance body to exclude the fixed front frame. The gripper approaches the exterior panel face and follows bounded hinge-frame waypoints. A fresh angle estimate after a motion plateau can initiate withdrawal; a second view after retreat checks closure.

After drawer contact, the next grasp may require substantial wrist motion. The controller compares equivalent jaw frames along a continuous inverse-kinematics path using the public robot model and bounds simultaneous translation and rotation. This is a local reachability heuristic, without collision checking. During bowl transport, it uses the observed drawer front and held-object footprint to move clear of the panel before lifting.

Contact guards combine position error, measured progress, and changes in end-effector force. One guard requires a minimum contact duration and bounded position error, plus either stalled error reduction or sufficient force change. This provides evidence of contact; a subsequent visual check assesses the resulting fixture configuration. The separation allows the controller to distinguish a mechanical stop from its estimate of task completion.

G. Action Generation and Transition Checks

For a geometric target pose (p^*, R^*) and gripper command g_t , the basic Cartesian servo emits

$$a_t = \text{clip}_{[-1,1]} \begin{bmatrix} (p^* - p_t)/s_p \\ \text{Log}(R^* R_t^T)^V / s_R \\ g_t \end{bmatrix}. \quad (6)$$

The rotation term is the axis-angle error between the current and target orientations. The general grasp–place scales are $s_p = 0.05$ m and $s_R = 0.25$ rad; cavity handling can select another rotation scale. Nominal pose tolerances of 8 mm and 0.12 rad gate transitions between motion phases. Specialized branches additionally bound translation or rotation during constrained motions.

Phase counters and geometric checks bound local recovery. The controller may release a rejected grasp, reacquire a visible handle, or execute a short clearance motion after stalled drawer interaction. Each stage supplies explicit targets, observations, and stopping reasons for inspection. These checks govern execution using the policy’s own measurements. The benchmark predicate is unavailable to both action selection and stopping, so local completion remains a separate outcome, evaluated in Section V-C.

IV. EXPERIMENTAL PROTOCOL

A. Tasks, Initial States, and Runtime

The evaluation contains $4 \times 10 \times 10 = 400$ episodes: Spatial, Object, Goal, and Long, ten tasks each, and official initial-state indices 0–9. Long is named `libero_10` in the implementation. Global random generators and the environment configuration use seed 7. The environment adapter preserves or replays the task reset stream when starting at a later initial-state index, because resetting can affect fixture configuration beyond the saved joint state.

We use two 256×256 RGB-D cameras and a Panda with operational-space pose control. A ten-step settling period precedes scored actions. The Spatial/Object/Goal/Long action budgets are 220/280/300/520, matching the numerical budgets in the OpenVLA evaluation script [12]. Our environment seed and success-handling logic differ from that script; its environment utility uses seed 0. The full run uses Ubuntu 24.04 under WSL2, Python 3.12.14, PyTorch 2.11.0 with CUDA 13.0, MuJoCo 3.3.2 [13], robosuite 1.4.0 [14], and hf-libero 0.1.4 on one RTX 3090.

All 400 episodes are newly executed against one immutable source snapshot. Independent task processes run under a dispatcher, initially with four workers and then six. Each process starts from the declared seed and evaluates the ten initializations of one task in index order. Before evaluation, all simulator assets and model files are checked against pinned SHA256 hashes. The source manifest, effective configuration, episode coverage, and scoring mode are validated after the full run. The task initializations include states used during skill development; this evaluation does not measure held-out generalization.

B. External Scoring and Internal Completion

Let q_t be the official success predicate observed by the evaluator. With policy-controlled stopping time τ capped at budget H , the recorded outcome is

$$Y = \bigvee_{t=1}^{\tau} q_t \vee q_{\text{final}}, \quad \tau \leq H. \quad (7)$$

The final query also handles a zero-action stop. Reward and success do not enter the policy observation. The evaluator continues after a success signal until the controller requests stopping or the action budget expires. The score therefore represents *ever reaching* the predicate during this rollout, not necessarily satisfying it at the final frame.

Each record additionally stores terminal controller state, step count, elapsed time, failure message, and geometric

TABLE III

EXTERNAL EVER-SUCCESS COUNTS AND EXECUTION COST.

[†]ELAPSED-TIME MEDIANS INCLUDE EXECUTION OVERHEAD UNDER CONCURRENT EVALUATION. ALL COUNTS, STEPS, AND TIMES COME FROM THE SAME FULL RUN.

Suite	Success	SR (%)	Mean steps	Median $s^{\dagger}$
Spatial	93/100	93.0	147.2	39.6
Object	98/100	98.0	195.4	32.5
Goal	98/100	98.0	179.9	41.3
Long	87/100	87.0	323.6	58.7
Overall	376/400	94.0	211.6	39.9

traces. The reported success rate is $\hat{p} = N^{-1} \sum_i Y_i$. Equal task sample counts make the overall episode average equal to the macro-average over tasks in the completed run. All planned episodes, including unsuccessful rollouts, enter the reported statistics. Controller-internal retries remain within one episode.

C. Scope of the Evidence

The completed campaign passes its record-coverage validation: 400 unique episodes, fixed configuration, existing recorded videos, and matching deployed source checksums.

Tables are generated from hashed original records. Published values are stored separately with their cited sources. Per-task counts expose the sample size behind each rate.

V. RESULTS AND ANALYSIS

A. Task Outcomes

The frozen system records 376 successes in 400 episodes (94.0%; Table III).

The suite rates are 93.0% for Spatial, 98.0% for Object, 98.0% for Goal, and 87.0% for Long. Long contains 13 of the 24 external failures (54.2%), despite contributing one quarter of the episodes. The overall rate therefore masks lower reliability on the Long suite.

The task distribution reveals variation hidden by the overall rate (Fig. 2). 25 tasks have ten successful recorded trials. Long 09, placing a mug inside a microwave and closing it, records 8/10; Long 08, placing both moka pots on the stove, records 8/10. These tasks require confined access, sequential placement, or fixture interaction. Drawer-related tasks also differ: Goal 03, opening the top drawer and placing a bowl inside, records 9/10; Spatial 04, retrieving a bowl from a top drawer, records 7/10; and Long 03, placing a bowl in a bottom drawer and closing it, records 7/10. The variation motivates examining individual execution stages rather than attributing all failures to a task category.

B. Execution Traces and Visual Examples

Figure 3 aligns recorded images with the executed action count and logged control phase. These labels expose which stage produced the motion, while the underlying traces retain its geometric targets and accepted observations.

The successful Object 00 example completes in 136 actions. In the Goal 03 example, the controller enters pregrasp

TABLE IV

TERMINAL CONTROLLER STATE AND INDEPENDENTLY RECORDED EXTERNAL EVER-SUCCESS. A CONTROLLER FAILURE CAN FOLLOW AN EARLIER BENCHMARK SUCCESS.

Controller terminal state	External success	External failure
Succeeded	341	6
Failed	24	18
Timeout	11	0

at action count 111 and remains in that phase for 161 actions before stopping at 272, with 28 actions left in the episode budget. The timeout locates the unsuccessful stage after drawer interaction; it does not establish whether contact, an unreachable target, or estimation error caused the stall.

C. Local Completion and External Success

Table IV crosses local completion with external ever-success. Of 347 locally completed episodes, 341 reach the official predicate (98.3%), while 6 never do. Conversely, 35 episodes reach external success without controller completion. The total disagreement is 41/400 (10.2%).

Local checks can therefore accept a rollout that never satisfies the goal, or continue demanding a geometric relation after the goal has been reached. The latter group includes later local failures and global-budget timeouts. This cross-tabulation exposes the scope of local verification: an ever-success label does not establish that the controller recognizes or preserves the goal until stopping.

The records retain neither first-success time nor a separate final-state label. They therefore cannot determine how continued execution changes final-state success or how much runtime an early-stop evaluator would save.

D. Remaining Failure Categories

Table V groups all 24 external failures by terminal state and message. Phase timeouts form the largest group. Grouping prioritizes controller completion with external failure and global budget exhaustion before classifying the remaining termination messages. All 24 externally unsuccessful episodes stop before their global action cap, leaving at least 13 actions unused. Conversely, all 11 global-budget timeouts have an external success label. The observed external failures therefore end at local stopping conditions, which motivates examining phase targets and termination checks.

These categories locate where execution stopped. They do not identify physical root causes: a timeout can arise from an inaccessible target, occlusion, calibration error, or contact resistance, and a rejected verification can reflect either placement error or uncertain perception. Per-episode geometric traces and recordings supply the additional evidence needed to investigate a particular failure.

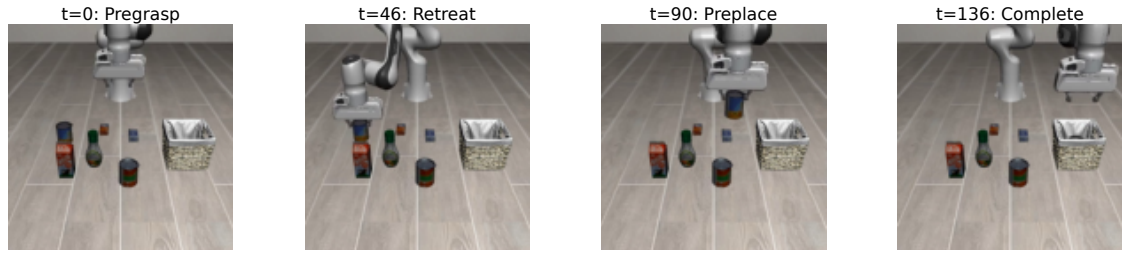
E. Model Requirements and Published Results

Table VI separates neural model size from the use of robot demonstrations for action generation. ANCHOR has 0.172B parameters in its frozen detector and no learned

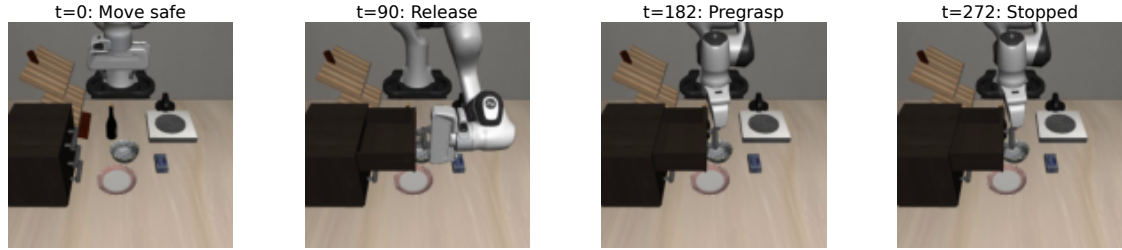
Spatial	10/10	8/10	10/10	9/10	7/10	9/10	10/10	10/10	10/10	10/10
Object	10/10	9/10	10/10	10/10	9/10	10/10	10/10	10/10	10/10	10/10
Goal	10/10	10/10	10/10	9/10	10/10	10/10	9/10	10/10	10/10	10/10
Long	10/10	9/10	9/10	7/10	7/10	10/10	9/10	10/10	8/10	8/10
	00	01	02	03	04	05	06	07	08	09
	Official task index (within each suite)									

Fig. 2. External successes per ten trials. Columns identify different tasks in each suite; darker cells indicate higher success.

Object 00 / init 00: external success



Goal 03 / init 00: phase timeout



Goal 03 controller trace (actions)



Fig. 3. Actual fixed-camera frames from the local full evaluation. Top: *pick up the alphabet soup and place it in the basket* (Object 00, initial state 00). Bottom: *open the top drawer and put the bowl inside* (Goal 03, initial state 00). Four uniformly spaced frames are shown. t counts executed actions, and phase labels use the last logged transition at or before t . The bottom bar shows the failed episode’s earlier actions, final active phase, and unused global budget. Frames are vertically flipped for upright display; left/right is preserved.

TABLE V

OUTCOME GROUPS AMONG EXTERNAL FAILURES. CATEGORIES ARE DERIVED FROM RECORDED CONTROLLER MESSAGES WITH A FIXED PRECEDENCE RULE.

Recorded outcome group	Episodes
Phase timeout	13
Controller completion / external failure	6
Grasp / retention rejection	2
Other controller failure	2
Sensor verification rejection	1

action network. The FP32 weight file occupies 689 MB (decimal). The reported SmolVLA and 7B OpenVLA model classes have approximately 2.6 and 40.6 times as many

parameters, respectively. Our count includes both visual and text encoders; the VLA sizes are rounded descriptions from the source papers.

The selected SmolVLA simulation model starts from a VLM without robotics pretraining but trains its action policy on LIBERO demonstrations [2]. ANCHOR relies on pretrained perception, asset priors, and manual skill development; its parameter count quantifies only the neural component.

Table VII provides literature context for these requirements. External rows retain the source papers’ reported LIBERO means and averages; ANCHOR is the only system evaluated in this study.

The OpenVLA study uses filtered demonstrations and

TABLE VI

MODEL SIZE AND ROBOT-DEMONSTRATION USE FOR ACTION-POLICY TRAINING. VLA SIZES ARE ROUNDED VALUES FROM THE CITED PAPERS; ANCHOR COUNTS ALL UNIQUE PARAMETERS IN ITS DEPLOYED DETECTOR. “NONE” REFERS TO ACTION-POLICY TRAINING, NOT TO PERCEPTION PRETRAINING OR MANUAL SKILL DEVELOPMENT.

Method	Parameters	Policy training
SmolVLA [2]	0.45B	Robot demos
OpenVLA [1]	7B	Robot demos
OpenVLA-OFT [3]	7B	Robot demos
ANCHOR	0.172B	None

reports averages over three random seeds, with 500 trials per suite and seed [1]. SmolVLA trains across all 40 tasks and evaluates ten trials per task [2]; OpenVLA-OFT uses filtered demonstrations and evaluates 500 trials per suite in the selected setting [3]. Our system uses calibrated RGB-D, public asset priors, ten official initializations per task, and the ever-success rule in Eq. (7).

The comparison describes different allocations of prior knowledge: learned action policies use robot demonstrations, while ANCHOR uses calibrated geometry and manually designed skills. These sensing, training, and protocol differences preclude attributing differences in success rate to the action representation or neural model size. Parameter counts also provide no measurement of relative latency or peak memory.

F. Execution Cost

Long episodes use 323.6 actions on average, compared with 195.4 for Object (Table III). These counts extend to the controller’s stopping condition or the budget, so they measure the cost of executed behavior rather than time to first success.

The median newly executed episode takes 39.9 seconds; dispatcher wall time is 62.7 minutes.

The records contain 0 runtime exceptions. Elapsed times include perception, simulation, rendering, and video overhead under the local hardware and concurrent scheduling. They describe this evaluation environment and do not measure real-robot control frequency.

VI. DISCUSSION AND LIMITATIONS

The evaluation characterizes geometric execution under explicit asset and sensing assumptions. Stored targets, accepted geometry, and transition checks make the controller’s decisions available for inspection. The difference between local completion and external success shows where this operational interpretation of a task can diverge from the benchmark predicate. In this run, external failures arise before the global action cap; the recorded phase targets and stopping checks provide concrete locations for further study.

a) Prior knowledge and development effort: The system’s compact neural component relies on calibrated RGB-D, public asset textures and dimensions, robot geometry, a constrained grammar, and manually developed skills.

Removing demonstration-based action-policy training shifts work to these representations and procedures. Parameter count and weight-file size quantify the detector, but do not quantify total engineering cost, peak memory, or end-to-end latency. Debugging effort was not compared with that of a learned policy.

b) Evaluation scope: The study covers known assets and instruction forms in one simulated embodiment. It does not test unseen assets, unrestricted language, perturbed calibration, or physical hardware. LIBERO was introduced for lifelong robot learning [7]; the present evaluation characterizes manipulation execution and does not measure transfer, continual learning, or forgetting. The initializations include states used during skill development, so the full run measures execution on the declared task set. No component ablations establish the individual contribution of the gallery, offset handling, or local guards. A fixed-controller evaluation on held-out conditions and ablations under matched sensing would be needed to test robustness and separate those effects.

c) Execution and measurement limits: Clipped commands, pose tolerances, and bounded retries are operational controls, not guarantees of tracking accuracy, collision avoidance, or contact safety. Small task samples can conceal rare failures, and the observed disagreement between controller and evaluator shows why stopping conventions matter. Recording first-success time and final-state success separately would distinguish goal attainment from goal preservation.

d) Reproducibility: The artifacts preserve original episode records, controller and resource checksums, and table/figure generation scripts. Observation restrictions are supported by interface and code inspection.

VII. CONCLUSION

We presented ANCHOR, an asset-informed system that connects language instructions to geometric manipulation skills through observed scene geometry, carried-object offsets, and local transition checks. One frozen 172M-parameter detector supplies its neural component, with no robot-demonstration training of an action policy.

The complete evaluation records 376/400 external successes (94.0%).

These results support inspectable geometric execution with known assets. Generalization and the causal benefit of individual components require further evaluation.

ACKNOWLEDGMENTS AND AI DISCLOSURE

OpenAI GPT-6-Astra (via Codex) assisted in drafting and editing the text in all sections, preparing the LaTeX source, and writing scripts for the numerical tables and figures. Results for ANCHOR are computed from existing simulator evaluation records; comparison values are taken from the cited publications. The rollout images are extracted from our recordings; no generative image model was used.

TABLE VII

LIBERO SUCCESS RATES (%). PUBLISHED ROWS CONTAIN THE REPORTED MEANS FROM OPENVLA v3, TABLE 12 [1], SMOLVLA v1, TABLE 2 [2], AND OPENVLA-OFT v2, TABLE I [3] (ARXIV VERSIONS); THEY ARE NOT RERUNS BY US. OPENVLA-OFT USES ITS FULL SETTING. THE FINAL ROW IS MEASURED IN THIS WORK. ALL METHODS RECEIVE LANGUAGE; THEIR SENSING, TRAINING, AND PROTOCOLS DIFFER.

Method	Sensing and action-policy setup	Spatial	Object	Goal	Long	Average
<i>Published results (reported means)</i>						
Diffusion Policy [1]	Single RGB; policy trained from scratch per suite	78.3	92.5	68.3	50.5	72.4
Octo [1]	Single RGB; pretrained policy fine-tuned per suite	78.9	85.7	84.6	51.1	75.1
OpenVLA [1]	Single RGB; pretrained policy fine-tuned per suite	84.7	88.4	79.2	53.7	76.5
SmolVLA (0.45B) [2]	RGB/state; VLM-initialized policy trained across suites	90.0	96.0	92.0	71.0	87.3
OpenVLA-OFT [3]	Two RGB views + state; policy fine-tuned per suite	97.6	98.4	97.9	94.5	97.1
<i>Measured in this work</i>						
ANCHOR	Two RGB-D/state; asset priors, geometric skills	93.0	98.0	98.0	87.0	94.0

REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, *et al.*, “OpenVLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246v3*, 2024, appendix E, Table 12.
- [2] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, *et al.*, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844v1*, 2025, table 2, 0.45B simulation model.
- [3] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” in *Proceedings of Robotics: Science and Systems*, 2025, results from arXiv:2502.19645v2, Table I.
- [4] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” in *IEEE International Conference on Robotics and Automation*, 2023, pp. 9493–9500.
- [5] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composable 3D value maps for robotic manipulation with language models,” in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 540–562.
- [6] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, “ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation,” in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 4573–4602.
- [7] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 44 776–44 791.
- [8] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, “Integrated task and motion planning,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, no. 1, pp. 265–293, 2021.
- [9] L. Manuelli, W. Gao, P. Florence, and R. Tedrake, “kPAM: KeyPoint affordances for category-level robotic manipulation,” in *Robotics Research*, ser. Springer Proceedings in Advanced Robotics, vol. 20, 2022, pp. 132–157.
- [10] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, *et al.*, “Do as I can, not as I say: Grounding language in robotic affordances,” in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205, 2023, pp. 287–318.
- [11] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, *et al.*, “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *Computer Vision – ECCV 2024*, ser. Lecture Notes in Computer Science, vol. 15105, 2025, pp. 38–55.
- [12] OpenVLA Contributors, “LIBERO evaluation script and environment utilities,” <https://github.com/openvla/openvla/tree/c8f03f48af69/experiments/robot/libero>, commit c8f03f48af69, accessed September 14, 2026.
- [13] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.
- [14] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu, “robosuite: A modular simulation framework and benchmark for robot learning,” *arXiv preprint arXiv:2009.12293v3*, 2025.